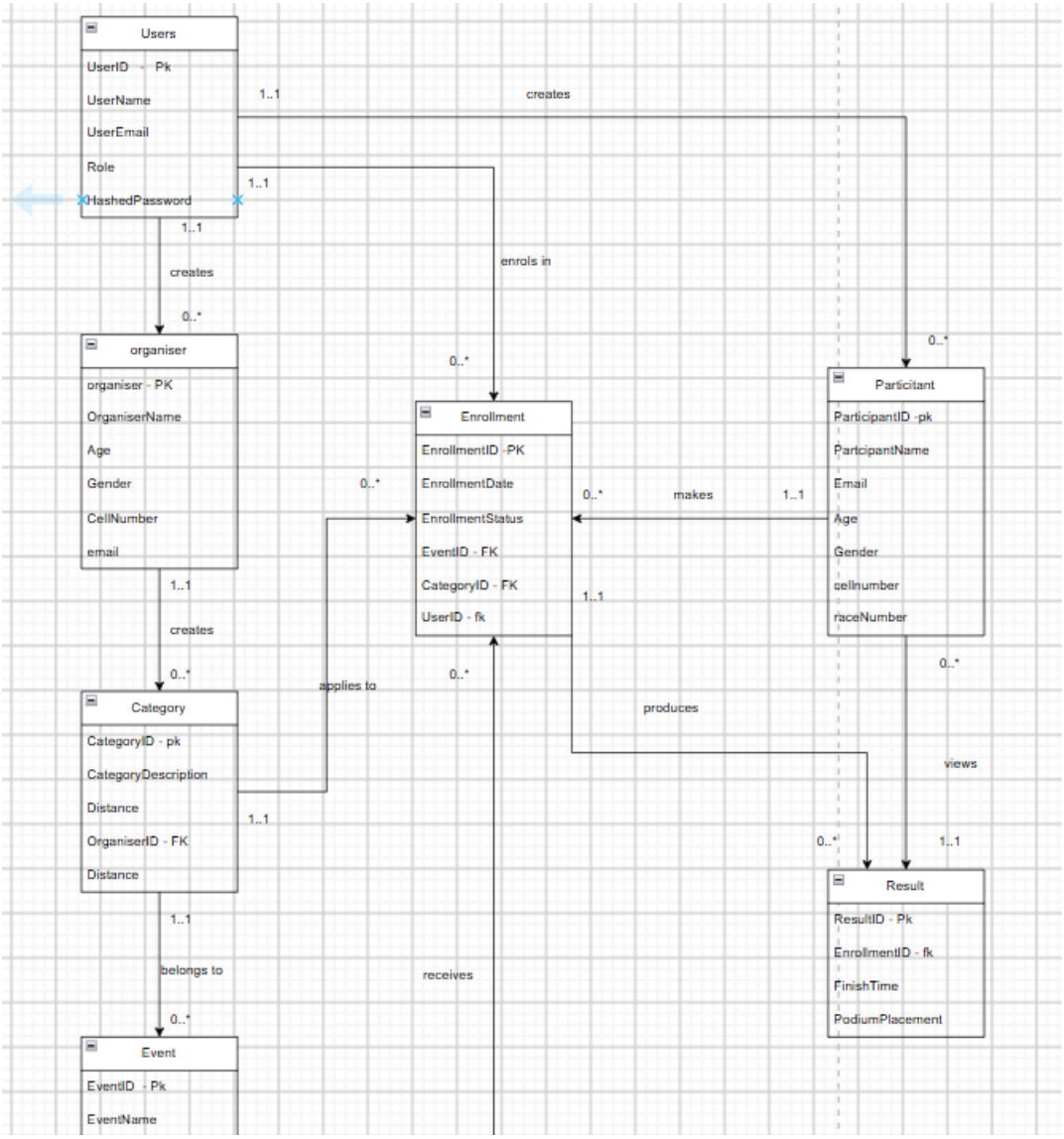
Section A

The ERD





(draw.io, 2026)

Business rules

1. User Account Management, A User must register and log in to access the system. Each User record stores authentication details and role information.
2. Role Assignment, A User can assume one role only — either Organiser or Participant — which determines their system privileges.
3. Category Definition, An Organiser can define multiple Categories to group participants by race type or age group. Each Category is created by one Organiser.
4. Event Category Relationship, Each Event can include multiple Categories, but every Category belongs to one Event.
5. Event Ownership, An Organiser can create and manage many Events, while each Event is hosted by a single Organiser.
6. Participant Enrollment, A Participant can enrol in multiple Events. Each Enrollment record links one Participant to one Event and one Category.
7. Enrollment Tracking, Each Enrollment stores the date and status of participation (Confirmed, Pending, or Cancelled) for audit and management purposes.
8. Result Recording, Each Enrollment may produce one Result, capturing the participant's finish time and podium placement.
9. Result Visibility, A Participant can view all their Results after events conclude. Many Results can be associated with one Participant.
10. Data Persistence, The system must maintain historical records of all Users, Events, Categories, Enrollments, and Results for reporting and tracking.

Section B

API endpoint table

Http Methods	Route	Description	Role Required	Request Body	Expected Response
POST	/api/Authe/register	Register a new user (Organiser or Participant)	Public	{UserName, UserEmail, Role, password}	200 OK – User registered successfully
POST	/api/Authe/login	login existing user	Public	{UserEmail, password}	200 OK – Login successful / 401 Unauthorized
POST	/api/Authe /logout	Clears all session data to log the user out securely.	Participant/Organiser	none	`200 OK – {"message": "Logged out successfully and session cleared."}`
GET	/api/Authe/profile/participant/{userId}	View participant profile	Participant	none	200 OK – Profile data / 404 Not Found
PUT	/api/Authe/profile/participant/update	Update participant profile	Participant	{userId, age, gender, cellnumber}	200 OK – Profile updated
GET	/api/Authe/profile/organiser/{userId}	`View organiser profile`	Organiser	none	200 OK – Profile data / 404 Not Found
PUT	/api/Authe/profile/organiser/update	Update organiser profile	Organiser	{userId, age, gender, cellnumber}	200 OK – Profile updated
GET	/api/Event/all	View all events	public	none	200 OK – List of events

POST	/api/Event/create	Create a new event	Organiser	{eventName, date, location, categoryId}	200 OK – Event created successfully
PUT	/api/Event/update	Update event details	Organiser	{eventId, eventName, date, location}	200 OK – Event updated successfully
DELETE	/api/Event/delete/{eventId}	Delete an event	Organiser	none	200 OK – Event deleted successfully
POST	/api/Category/create	Create a new category	Organiser	{categoryName, ageRange, distance}	200 OK – Category created successfully
GET	/Api/category/all	View all categories	public		200 OK – List of categories
POST	/api/Enrollment/create	Enroll participant in event	Participant	{eventId, categoryId, participantId}	200 OK – Enrollment created successfull
GET	/api/Enrollment/event/{eventId}	View all enrollments for an event	Organiser		200 OK – List of enrollments
POST	/api/Result/create	Capture race results	Organiser	{enrollmentId, finishTime, podiumPlacement}	200 OK – Result captured successfully
GET	/api/Result/participant/{userId}	View participant results	participant		200 OK – List of results

Section C

Database Part

```
--Crating a database

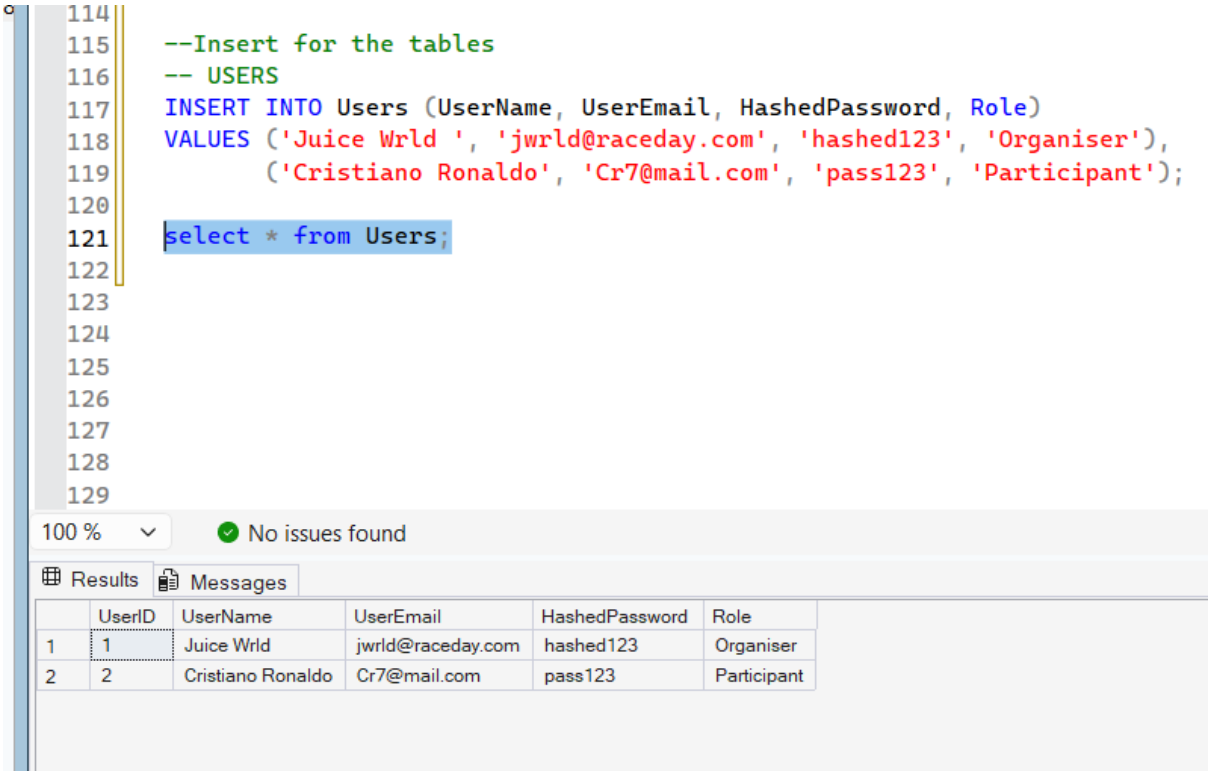
create database Prog;


use Prog;
```

-- creating users table

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY IDENTITY(1,1),
    UserName VARCHAR(100) NOT NULL,
    UserEmail VARCHAR(100) UNIQUE NOT NULL,
    HashedPassword VARCHAR(255) NOT NULL,
    Role VARCHAR(20) NOT NULL -- role = Organiser or Participan
);
```

Output



-- organiser tAble

```
CREATE TABLE Organiser (
    OrganiserID INT PRIMARY KEY IDENTITY(1,1),
    OrganiserName VARCHAR(100) NOT NULL,
    CellNumber VARCHAR(20),
    Email VARCHAR(100),
    UserID INT NOT NULL,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
```

);

```
123    -- Organiser
124    INSERT INTO Organiser (OrganiserName, CellNumber, Email, UserID)
125    VALUES ('Juice Wrld', '0821112222', 'jwrld@raceday.com', 1);
126
127
128    select * from Organiser;
129
130
131
```

100 % No issues found

Results Messages

	OrganiserID	OrganiserName	CellNumber	Email	UserID
1	1	Juice Wrld	0821112222	jwrld@raceday.com	1

-- category table

CREATE TABLE Category (

CategoryID INT PRIMARY KEY IDENTITY(1,1),

CategoryDescription VARCHAR(100) NOT NULL,

Distance VARCHAR(50),

Item1 VARCHAR(50),

OrganiserID INT NOT NULL,

FOREIGN KEY (OrganiserID) REFERENCES Organiser(OrganiserID)

);

```sql
-- Catergory
INSERT INTO Category (CategoryDescription, Distance, Item1, OrganiserID)
VALUES ('10km Run', '10km', 'Under 20', 1);

select * from Category;
```

100 % No issues found

Results Messages

	CategoryID	CategoryDescription	Distance	Item1	OrganiserID
1	1	10km Run	10km	Under 20	1

-- event table

```sql
CREATE TABLE Event (
    EventID INT PRIMARY KEY IDENTITY(1,1),
    EventName VARCHAR(100) NOT NULL,
    EventDescription VARCHAR(255),
    EventType VARCHAR(50),
    EventDate DATE NOT NULL,
    PlaceOfTheEvent VARCHAR(100),
    CategoryID INT NOT NULL,
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);
```

```sql
-- EVENT
INSERT INTO Event (EventName, EventDescription, EventType, EventDate, PlaceOfTheEvent, CategoryID)
VALUES ('Joburg Marathon', 'Annual city marathon', 'Run', '2026-10-15', 'Johannesburg', 1);

Select * from Event;
```

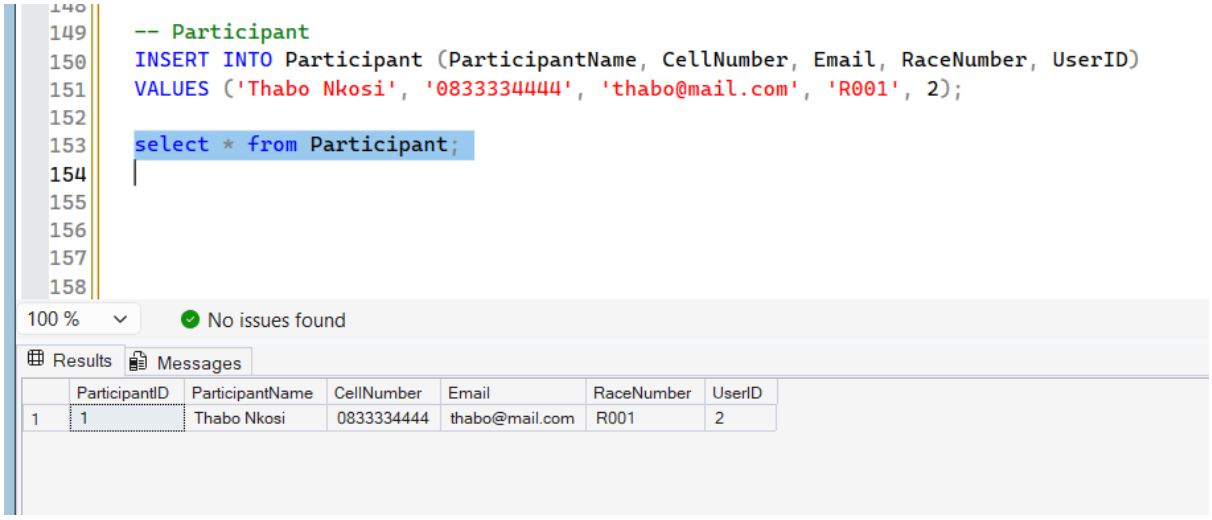
100 % No issues found

Results Messages

	EventID	EventName	EventDescription	EventType	EventDate	PlaceOfTheEvent	CategoryID
1	1	Joburg Marathon	Annual city marathon	Run	2026-10-15	Johannesburg	1

-- participant table

```
CREATE TABLE Participant (
    ParticipantID INT PRIMARY KEY IDENTITY(1,1),
    ParticipantName VARCHAR(100) NOT NULL,
    CellNumber VARCHAR(20),
    Email VARCHAR(100),
    RaceNumber VARCHAR(20),
    UserID INT NOT NULL,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);
```



New Additions (Attributes) added to the Participant and Organiser Tables

```
-- Add Age and Gender to Organiser
Alter table Organiser
Add Age Int NULL,
Gender VARCHAR(20) NULL;

-- Add Age and Gender to Participant
Alter Table Participant
Add Age Int NULL,
Gender VARCHAR(20) NULL;
```

-- enrollment tables

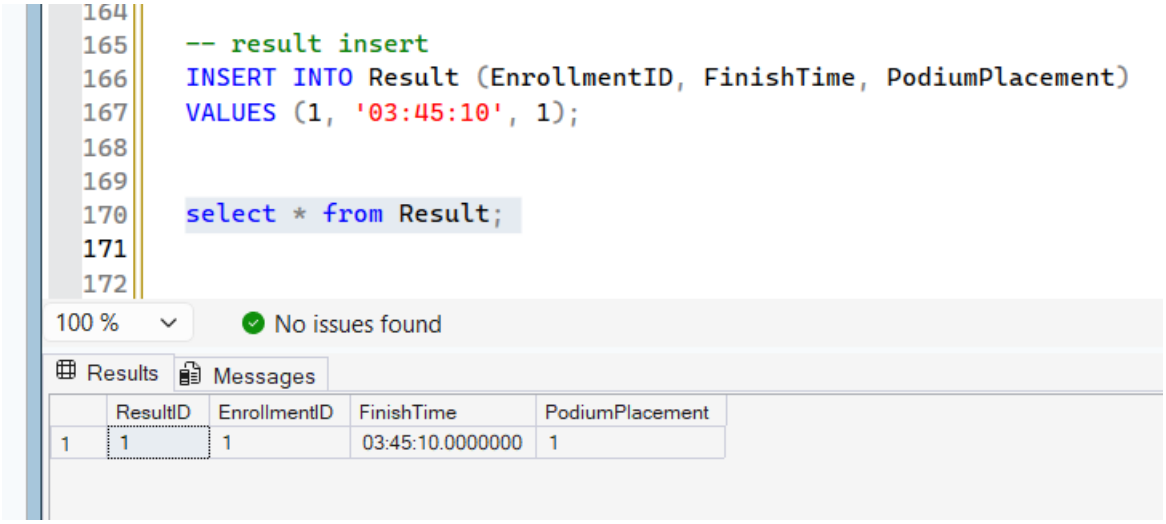
CREATE TABLE Enrollment (

```
    EnrollmentID INT PRIMARY KEY IDENTITY(1,1),
    EnrollmentDate DATE NOT NULL,
    EnrollmentStatus VARCHAR(50) NOT NULL,
    UserID INT NOT NULL,
    EventID INT NOT NULL,
    FOREIGN KEY (UserID) REFERENCES Users(UserID),
    FOREIGN KEY (EventID) REFERENCES Event(EventID)
);
```

```
-- enrollment
insert into Enrollment (EnrollmentDate, EnrollmentStatus, UserID, EventID)
VALUES ('2026-09-01', 'Confirmed', 2, 1);

select * from Enrollment;
```

```
-- RESULT TABLE
CREATE TABLE Result (
    ResultID INT PRIMARY KEY IDENTITY(1,1),
    EnrollmentID INT NOT NULL,
    FinishTime TIME,
    PodiumPlacement INT,
    FOREIGN KEY (EnrollmentID) REFERENCES Enrollment(EnrollmentID)
);
```




```
--Crating a database
create database Prog;

use Prog;

-- creating users tablCREATE TABLE Users (
   UserID INT PRIMARY KEY IDENTITY(1,1),
   UserName VARCHAR(100) NOT NULL,
   UserEmail VARCHAR(100) UNIQUE NOT NULL,
   HashedPassword VARCHAR(255) NOT NULL,
   Role VARCHAR(20) NOT NULL -- There are two roles, Participant and Organiser
);


-- Add Age and Gender to Organiser
Alter table Organiser
Add Age Int NULL,
Gender VARCHAR(20) NULL;

-- Add Age and Gender to Participant
Alter Table Participant
Add Age Int NULL,
Gender VARCHAR(20) NULL;



-- organiser tAble
CREATE TABLE Organiser (
   OrganiserID INT PRIMARY KEY IDENTITY(1,1),
   OrganiserName VARCHAR(100) NOT NULL,
```

```
    CellNumber VARCHAR(20),
    Email VARCHAR(100),
    UserID INT NOT NULL,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);




-- category table
CREATE TABLE Category (
    CategoryID INT PRIMARY KEY IDENTITY(1,1),
    CategoryDescription VARCHAR(100) NOT NULL,
    Distance VARCHAR(50),
    Item1 VARCHAR(50),
    OrganiserID INT NOT NULL,
    FOREIGN KEY (OrganiserID) REFERENCES Organiser(OrganiserID)
);

--deleting unwanted column item1
ALTER TABLE Category
DROP COLUMN Item1;









-- event table
CREATE TABLE Event (
    EventID INT PRIMARY KEY IDENTITY(1,1),
    EventName VARCHAR(100) NOT NULL,
    EventDescription VARCHAR(255),
    EventType VARCHAR(50),
    EventDate DATE NOT NULL,
    PlaceOfTheEvent VARCHAR(100),
    CategoryID INT NOT NULL,
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);

--adding OrganiserId in the event table
ALTER TABLE Event
ADD OrganiserID INT NOT NULL
    CONSTRAINT DF_Event_Organiser DEFAULT 1;

ALTER TABLE Event
ADD CONSTRAINT FK_Event_Organiser FOREIGN KEY (OrganiserID)
REFERENCES Organiser(OrganiserID);
```

```sql
-- participant table
CREATE TABLE Participant (
   ParticipantID INT PRIMARY KEY IDENTITY(1,1),
   ParticipantName VARCHAR(100) NOT NULL,
   CellNumber VARCHAR(20),
   Email VARCHAR(100),
   RaceNumber VARCHAR(20),
   UserID INT NOT NULL,
   FOREIGN KEY (UserID) REFERENCES Users(UserID)
);
```

```sql
-- enrollment tables
CREATE TABLE Enrollment (
   EnrollmentID INT PRIMARY KEY IDENTITY(1,1),
   EnrollmentDate DATE NOT NULL,
   EnrollmentStatus VARCHAR(50) NOT NULL,
   UserID INT NOT NULL,
   EventID INT NOT NULL,
   FOREIGN KEY (UserID) REFERENCES Users(UserID),
   FOREIGN KEY (EventID) REFERENCES Event(EventID)
);

--adding Category id
ALTER TABLE Enrollment
ADD CategoryID INT NOT NULL DEFAULT 1
FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID);
```

```sql
-- Result table
CREATE TABLE Result (
   ResultID INT PRIMARY KEY IDENTITY(1,1),
   EnrollmentID INT NOT NULL,
   FinishTime TIME,
   PodiumPlacement INT,
   FOREIGN KEY (EnrollmentID) REFERENCES Enrollment(EnrollmentID)
);
```

```sql
--Insert for the tables
-- USERS
INSERT INTO Users (UserName, UserEmail, HashedPassword, Role)
VALUES ('Juice Wrld ', 'jwrld@raceday.com', 'hashed123', 'Organiser'),
     ('Cristiano Ronaldo', 'Cr7@mail.com', 'pass123', 'Participant');
```

```sql
select * from Users;

-- Organiser
INSERT INTO Organiser (OrganiserName, CellNumber, Email, UserID)
VALUES ('Juice Wrld', '0821112222', 'jwrld@raceday.com', 1);

SELECT * FROM Organiser WHERE OrganiserID = 1;

INSERT INTO Organiser (OrganiserName, CellNumber, Email, UserID)
VALUES ('Test Organiser', '0839998888', 'test@raceday.com', 9);




select * from Organiser;



-- Catergory
INSERT INTO Category (CategoryDescription, Distance, Item1, OrganiserID)
VALUES ('10km Run', '10km', 'Under 20', 1);

INSERT INTO Category (CategoryDescription, Distance, OrganiserID)
VALUES ('10km Run', '10km', 1);

select * from Category;

SELECT * FROM Category WHERE CategoryID = 3;

INSERT INTO Category (CategoryDescription, Distance, Item1, OrganiserID)
VALUES ('Half Marathon', '21km', 'Adults', 2);




-- EVENT
INSERT INTO Event (EventName, EventDescription, EventType, EventDate, PlaceOfTheEvent, CategoryID)
VALUES ('Joburg Marathon', 'Annual city marathon', 'Run', '2026-10-15', 'Johannesburg', 1);

Select * from Event;



-- Participant
INSERT INTO Participant (ParticipantName, CellNumber, Email, RaceNumber, UserID)
VALUES ('Thabo Nkosi', '0833334444', 'thabo@mail.com', 'R001', 2);

select * from Participant;



-- enrollment
insert into Enrollment (EnrollmentDate, EnrollmentStatus, UserID, EventID)
VALUES ('2026-09-01', 'Confirmed', 2, 1);
```

```sql
select * from Enrollment;


-- result insert
INSERT INTO Result (EnrollmentID, FinishTime, PodiumPlacement)
VALUES (1, '03:45:10', 1);


select * from Result;


SELECT * FROM Category WHERE CategoryID = 1;
SELECT * FROM Organiser WHERE OrganiserID = 1;
```

Reference

- Draw.io., 2026. Available at: Untitled Diagram - draw.io [Accessed 29 August 2026]